COLLEGE MINI PROJECT · FULL-STACK E-COMMERCE · LIVE

Kai Ruchi

A complete online shop for a homemade South Indian food business — deployed and running — and a plain-English explanation of every part of it.

The website	`kai-ruchi.vercel.app`
The API	`kai-ruchi-api.vercel.app`
The code	`github.com/niri2630/kai-ruchi`
Demo login	`meera@example.com / kairuchi123`

FRONTEND	BACKEND	DATABASE	MIGRATIONS	HOSTING
Next.js 16	FastAPI	PostgreSQL	Alembic	Vercel + Supabase

01 What you have actually built

> Kai Ruchi is a working online shop that is live on the internet right now. Not a design, not a mock-up. You can browse products, search them, put them in a basket, make an account, pay, and then watch your order move from "placed" to "delivered".

It sells the food a single home kitchen in Udupi makes: masalas, pickles, fresh batters, fried snacks and sweets. Fourteen products across five categories.

What a visitor can do

- **Browse** — a home page, five category pages, and a page for every product.
- **Search** — type a word and matching products appear instantly as you type.
- **Filter and sort** — by category, spice level, vegetarian or not, in stock or not, and by price or rating.
- **Add to a cart** — which works before you log in, and follows you into your account when you do.
- **Make an account** — sign up, sign in, edit details, save delivery addresses.
- **Check out and pay** — on a simulated payment screen (see section 8).
- **Track the order** — a six-step timeline showing where the parcel is.
- **Write reviews** — marked "verified" automatically if that account really bought the product.
- **Contact the shop** — a form that saves the message into the database.

The size of it, in numbers

PART	COUNT	PART	COUNT
Pages on the website	**17**	Database tables	**12**
API endpoints	**36**	Products in the catalogue	**14**
Lines of frontend code	**≈ 8,400**	Lines of backend code	**≈ 3,100**
Photographs	**21**	Videos	**5**

SAY THIS OUT LOUD

The one-sentence description

"It is a full-stack e-commerce web application. The frontend is built with Next.js and React, the backend is a REST API built with FastAPI in Python, and the data is stored in a PostgreSQL database with Alembic managing schema changes. It is deployed on Vercel with the database hosted on Supabase."

What it deliberately does *not* have

There is **no admin dashboard**. That was a scope decision — it is a mini project, not a business. Two consequences follow, and a teacher may well ask about them:

- **Orders move on a timer.** A real shop has staff clicking "packed" and "shipped". Here, once an order is paid it walks itself along the tracking timeline over about 35 minutes so it can be demonstrated live.
- **Products are added by editing a seed file** (`backend/app/seed.py`) and re-running it, rather than through a screen.

02 The words you need to know

> Learn these and you can hold any conversation about this project. Everything else is detail.

WORD	WHAT IT ACTUALLY MEANS
Frontend	Everything the user sees and clicks — pages, buttons, images, animations. Runs inside the visitor's browser.
Backend	The program on a server that the frontend talks to. It does the thinking: checking passwords, calculating totals, saving orders.
Database	Where information is permanently stored. If the server restarts, the database still remembers everything.
API	The list of things the frontend may ask the backend for. Like a menu: "give me all products", "add this to my cart".
Endpoint	One item on that menu, e.g. `GET /api/products`.
REST	The convention for designing those menus using addresses plus verbs — GET (fetch), POST (create), PATCH (change), DELETE (remove).
JSON	The text format both sides use to exchange data: `{"name":"Mango Pickle","price":279}`.
Schema	The shape of the database — which tables exist and what columns they have.
Migration	A recorded change to that shape, so it can be repeated on any computer.
Deploy	Putting the project on the internet instead of only on your laptop.
Environment variable	A setting kept outside the code — passwords, database addresses. Never committed to GitHub.

JWT (JSON Web Token)

On login the backend gives your browser a long signed string. The browser sends it with every later request, so the server knows it is still you without asking for your password again.

ORM (Object-Relational Mapper)

A translator between Python code and database tables. Instead of writing SQL by hand you write `db.query(Product)` and the ORM — SQLAlchemy — writes the SQL for you.

Serverless function

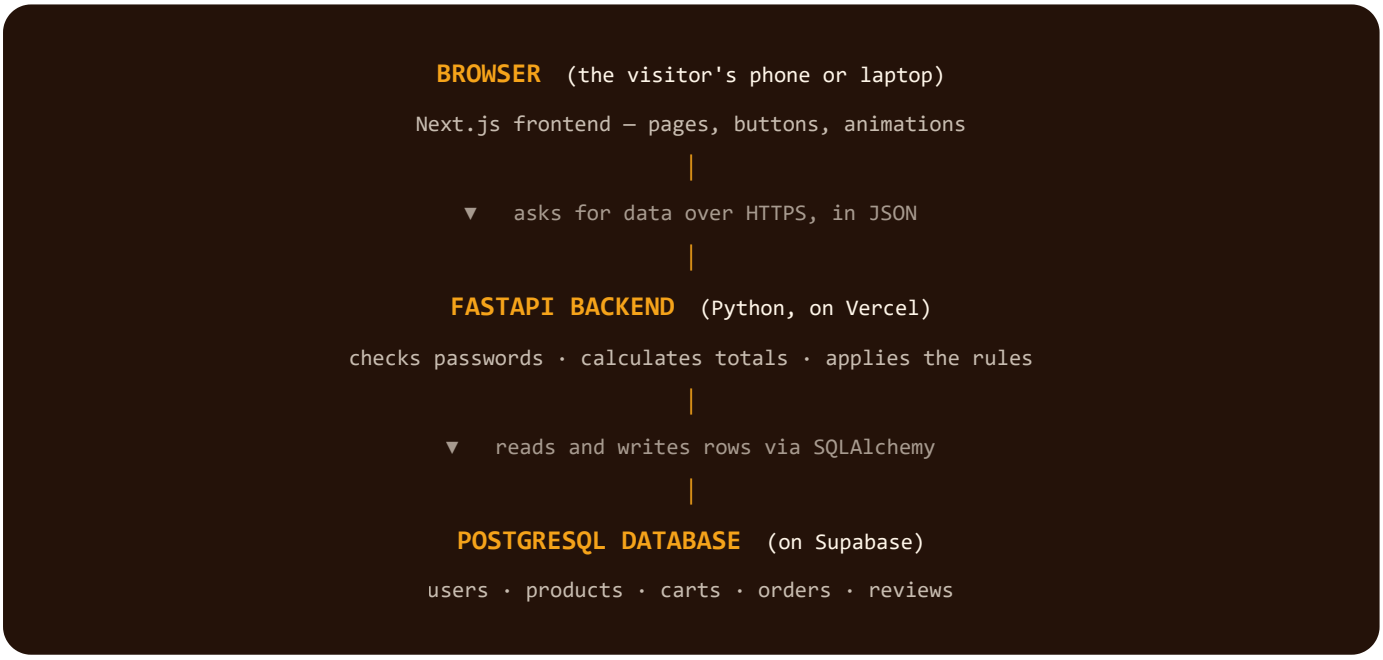
Code that runs only when a request arrives, then stops. You do not rent a server that idles all day. This is how the backend runs on Vercel.

Connection pooler

A middleman that keeps a set of database connections open and shares them out, so hundreds of short-lived functions do not each open their own.

03 The three pieces, and how they talk

> Every full-stack project is the same three boxes. Once you can draw this on a whiteboard, you understand the architecture.



Why keep them separate?

Because they have different jobs and different risks. The frontend can be rewritten without touching the data. The backend enforces rules the browser cannot be trusted with — a visitor could edit the page in their browser and set a price to ₹1, but the backend recalculates the real price before saving, so it does not matter.

SAY THIS OUT LOUD

"The frontend never trusts its own numbers. At checkout the browser sends only product IDs and quantities. The backend looks the real prices up in the database and recalculates the total itself, so the price cannot be tampered with."

A real example: what happens when you click "Add to cart"

1. You click the green **+** on a product card.
2. The frontend sends `POST /api/cart/items` with `{product_id: 5, quantity: 1}`.
3. The backend checks: **does this product exist? is it still in stock?** If only 3 remain and you asked for 4, it refuses with "Only 3 left in this batch."
4. If you are logged in it finds the cart on your account. If not, it uses a random token stored in your browser, so guests get a cart too.
5. It saves the row, then recalculates the subtotal, delivery fee (free over ₹799) and total.
6. It sends the whole updated cart back as JSON.
7. The frontend updates the badge, pops the number, and shows a toast: "Mango Pickle added".

04 Every technology, and why it was chosen

> A teacher will ask "why did you use X?" Here is a real answer for each, not just a description.

Frontend

TECHNOLOGY	WHAT IT DOES	WHY THIS ONE
Next.js 16	The React framework — routing, page structure, build system.	Every folder in `src/app` automatically becomes a page. It also deploys to Vercel with no configuration.
React 19	Builds the interface from reusable components.	A product card is written once and used on six pages.
TypeScript	JavaScript with type checking.	If the backend sends a price as text and the code treats it as a number, that is caught while writing, not by a user.
Tailwind CSS v4	Styling as small class names on the element.	No separate stylesheet to keep in sync; design tokens live in one file.
Framer Motion	All the animation.	It can animate elements as they leave the screen — plain CSS cannot animate something being removed.
Lenis	Smooth scrolling.	Makes the page glide, and lets other animations follow scroll position smoothly.
Zustand	Holds shared state — who is logged in, what is in the cart.	The navbar and cart page both need the cart; Zustand keeps one copy both read from.

Backend

TECHNOLOGY	WHAT IT DOES	WHY THIS ONE
FastAPI	The Python web framework providing the API.	It generates interactive API documentation automatically — open `/docs` and every endpoint is listed and testable. Excellent to show a teacher.
SQLAlchemy 2	The ORM.	Lets us write Python instead of raw SQL, and prevents SQL-injection by design.
Pydantic v2	Validates all incoming data.	A review with a rating of 9 is rejected before it reaches the database.
Alembic	Manages database migrations.	Schema history lives in files, so the same database can be rebuilt anywhere with one command.
bcrypt	Hashes passwords.	Passwords are never stored — only a one-way scramble.
python-jose	Creates and verifies JWT login tokens.	Signed with a secret key, so a token cannot be forged.

Hosting

SERVICE	WHAT IT DOES	WHY THIS ONE
PostgreSQL	Stores all the data.	Free, reliable, supports real relationships, exact decimal money values, and enum types for order status.
Supabase	Hosts that PostgreSQL online, free.	A database on your laptop is not reachable from the internet.
Vercel	Hosts both the website and the API.	Free tier, deploys straight from GitHub, and now runs Python backends as well as Next.js.

05 How the database is designed

> Twelve tables. Every one exists for a reason, and the reasons are the interesting part.

TABLE	HOLDS
users	Name, email, phone, and the hashed password.
addresses	Saved delivery addresses. One user can have many.
categories	The five shelves — masalas, pickles, fresh batters, snacks, sweets.
products	The fourteen items: price, stock, spice level, description, ingredients, photo.
carts	One open cart per user — or per guest, keyed by a random browser token.
cart_items	One row per product in a cart, with its quantity.
orders	A confirmed purchase: totals, payment status, delivery address.
order_items	What was actually in that order.
order_events	One row each time the order changes status — this is what the tracking timeline reads.
reviews	Star rating and text. One review per person per product.
contact_messages	Messages sent through the contact form.
alembic_version	Which migration has been applied. Alembic's own bookkeeping.

Three design decisions worth explaining

SAY THIS OUT LOUD

1. Why `order_items` copies the product name and price

It looks like duplicated data, and it is deliberate. If the price of Mango Pickle changes next month, an old invoice must still show what was actually paid. So the order stores its own copy. Same for the delivery address — editing a saved address must not rewrite where a past parcel was sent. This is called **snapshotting**.

SAY THIS OUT LOUD

2. Why there is a separate `order_events` table

The order could just have a "status" column, but then you would only know where it is *now*, not when it was packed or shipped. One row per change gives a full history, which is exactly what the tracking timeline displays.

SAY THIS OUT LOUD

3. Why guests get a cart too

Forcing people to sign up before adding to a basket loses sales. A guest cart is stored against a random token in the browser, and when that person signs in the backend **merges** it into their account cart instead of throwing it away.

Relationships, one line each

- A **category** has many **products**; a product belongs to one category.
- A **user** has many **addresses**, **orders** and **reviews**.
- A **cart** has many **cart_items**; each points at one product.
- An **order** has many **order_items** and many **order_events**.
- A **review** joins one user to one product — and the database enforces one review per person per product.

06 The API, endpoint by endpoint

> Thirty-six endpoints in nine files. You do not need to memorise them — you need the pattern.

METHOD	ADDRESS	WHAT IT DOES
POST	`/api/auth/signup`	Create an account, return a login token
POST	`/api/auth/login`	Sign in, return a login token
GET	`/api/auth/me`	Who am I? (requires the token)
GET	`/api/categories`	All five shelves, with product counts
GET	`/api/products`	Products, with filtering, sorting and paging
GET	`/api/products/suggest`	Type-ahead results for the search box
GET	`/api/products/{slug}`	Everything about one product
GET	`/api/cart`	My cart, with totals
POST	`/api/cart/items`	Add something to the cart
PATCH	`/api/cart/items/{id}`	Change a quantity
DELETE	`/api/cart/items/{id}`	Remove a line
POST	`/api/orders/checkout`	Turn the cart into an order
POST	`/api/payments/verify`	Confirm the payment
GET	`/api/orders/{number}`	One order plus its tracking timeline
POST	`/api/reviews/product/{slug}`	Write a review
POST	`/api/contact`	Send a message

DEMO TRICK

Open **kai-ruchi-api.vercel.app/docs** in a browser. FastAPI generates a live, clickable page listing every endpoint, which you can run right there against the real database. Far more convincing than a slide.

How the folders map to the API

```
backend/app/
  models/     the database tables, as Python classes
  schemas/    the shape of data going in and out (validation)
  routers/    the endpoints themselves, one file per topic
  services/   the business rules — cart totals, order timeline, payments
  core/       settings, database connection, security, shared helpers
  seed.py     the starting catalogue: 14 products, 6 demo users, 16 reviews
```

07 How the design was decided

> If asked "why does it look like this?", the answer is not "it looks nice". Every choice comes from the subject.

The colours

Taken off a South Indian kitchen shelf rather than picked from a palette tool:

NAME	COLOUR	WHERE IT COMES FROM
Chilli	`#e23e2e`	Byadgi chilli, and the kumkum at a doorway
Turmeric	`#f5a31a`	Fresh-ground turmeric
Banana leaf	`#1e7a54`	The plate a South Indian meal is served on
Indigo	`#4b2e83`	Temple indigo
Jasmine	`#fff6e9`	The page background — mallige flowers
Lamp soot	`#241209`	All the text

The four signature ideas

Glassmorphism

Cards are frosted glass because *a pickle jar is glass* — you shop by looking through it. It comes from the product, not from a trend.

Claymorphism

Buttons are soft and puffy with a hard bottom edge that compresses when pressed, so they feel like objects you push.

The masala mesh

Five spice-coloured blobs drift slowly behind the whole site. Every frosted surface is showing you that, blurred.

Kolam

The rice-flour pattern drawn on a South Indian doorstep each morning — used as the brand mark and the loading animation.

The typefaces

- **Bricolage Grotesque** — all headings. Modern and slightly irregular.
- **Instrument Sans** — body text. Quiet and very readable.
- **Baloo Thambi 2** — the "KaiRuchi" wordmark, chosen because it was designed alongside Tamil script, so the brand name sits in a face built for its own language.

The About page is set like a dictionary entry

The whole brand rests on one untranslatable word, so that page presents it as a lexicon entry — the headword enormous, a phonetic respelling, the languages it belongs to, and numbered senses. Everything after it is an editorial article with a drop cap and a hanging pull quote, rather than another stack of cards.

What was deliberately removed

Late in the build, six things were cut because they read as generic rather than designed: a "scroll ↓" cue, a bouncing arrow, two rows of count-up statistics, a row of trust badges, a looping all-caps product ticker, and a decorative divider between every section. **Knowing what you removed and why is a strong thing to be able to say.**

SAY THIS OUT LOUD

"Every animation respects the operating system's **prefers-reduced-motion** setting. If a visitor has asked their device to reduce motion — often for motion sickness — all of it switches off automatically. Accessibility was built in, not added afterwards."

The photographs

All 21 product and category photographs and the five films were generated with **Higgsfield** (Nano Banana Pro for stills, Seedance 2.5 for video). They arrived as 4K PNGs totalling about **200 MB**, which would make the site unusable, so a script converts them to WebP at the size they are actually displayed — **2.6 MB for the whole set, a 99% reduction**. Run it with `node scripts/optimise-images.mjs`.

08 The payment system — read this carefully

IMPORTANT

No real money moves, and no card details are ever collected.

The checkout opens a payment sheet clearly labelled **"Demo gateway · no real payment"**. It has **no fields for a card number, UPI ID or bank login** — deliberately, so there is nothing to enter and nothing to leak. Choosing a method and pressing pay simply tells our own backend the order was paid.

Why it is built this way

Taking real payments requires a registered business with a payment provider. This is a college project, so it simulates that step. There is also a **"Simulate a failed payment"** button, so the error path can be demonstrated too — not just the happy path.

But the real integration is written

The code for **Razorpay**, India's standard payment gateway, is fully written in `backend/app/services/payment_service.py`: creating a gateway order, and verifying the payment signature with HMAC-SHA256 so a fake confirmation cannot be forged. It stays switched off unless real API keys are configured.

SAY THIS OUT LOUD

"Payments run in mock mode by default. The Razorpay integration is implemented, including HMAC signature verification, but only activates when API keys are supplied. That way the whole checkout flow can be demonstrated end to end without a merchant account, and without ever handling card data."

Cash on delivery

The second option is genuine and needs no gateway at all — the order is confirmed immediately and marked to be paid to the courier.

09 Setting it up on your computer

> The site is already live, so this is only needed to develop or change it. Do these in order — each step assumes the one before worked.

Step 0 — install these four things first

INSTALL	GET IT FROM	HOW TO CHECK IT WORKED
Git	git-scm.com	`git --version`
Node.js (v20+)	nodejs.org	`node --version`
Python 3.12	python.org	`python --version`
PostgreSQL 16	postgresql.org	Remember the password you set during install

WATCH OUT

When installing Python on Windows, tick **"Add Python to PATH"** on the very first screen. Miss it and none of the commands below will work.

Step 1 — get the code

```
# Download the project from GitHub
git clone https://github.com/niri2630/kai-ruchi.git
cd kai-ruchi
```

Step 2 — create the database

```
psql -U postgres -c "CREATE DATABASE kairuchi;"
```

It will ask for the PostgreSQL password you chose during installation.

Step 3 — set up the backend

```
cd backend

# A private Python environment, so this project's packages
# do not clash with anything else on your computer
python -m venv .venv

# Install everything the backend needs
.venv/Scripts/python -m pip install -r requirements.txt

# Copy the settings template
cp .env.example .env
```

Open `backend/.env` and set the database line to your own password:

```
DATABASE_URL=postgresql+psycopg2://postgres:YOUR_PASSWORD@localhost:5432/kairuchi
```

Then build the tables and fill in the products:

```
# Create all the tables (Alembic running the migration)
.venv/Scripts/python -m alembic upgrade head

# Load 14 products, 5 categories, 6 demo users and 16 reviews
.venv/Scripts/python -m app.seed
```

Step 4 — set up the frontend

```
cd ../frontend
npm install
cp .env.example .env.local
```

Step 5 — run it

You need **two terminals open at once**. This is the part people get wrong: the backend and the frontend are two separate programs.

Terminal 1 — the backend

```
cd backend
.venv/Scripts/python -m uvicorn app.main:app --reload --port 8000
```

Leave running. Check `127.0.0.1:8000/docs`

Terminal 2 — the frontend

```
cd frontend
npm run dev
```

Leave running. Open `localhost:3000`

DEMO LOGIN

Email `meera@example.com`, password `kairuchi123`. All six demo accounts share it, and it works on the live site too. The sign-in page has a **"Fill in demo credentials"** button so you need not type it during a demo.

10 How it got onto the internet

> Three services, each doing one job. This section is worth understanding because deployment is where most student projects stop.

SERVICE	HOLDS	ADDRESS
GitHub	The code	`github.com/niri2630/kai-ruchi`
Vercel (project 1)	The website	`kai-ruchi.vercel.app`
Vercel (project 2)	The API	`kai-ruchi-api.vercel.app`
Supabase	The PostgreSQL database	region `ap-southeast-1` (Singapore)

The settings that make it work

None of these live in the code — they are **environment variables**, kept in each Vercel project's settings so they never appear on GitHub.

VARIABLE	SET ON	WHAT IT IS
`NEXT_PUBLIC_API_URL`	the website	Where to find the API
`DATABASE_URL`	the API	Where to find the database, with the password
`SECRET_KEY`	the API	Used to sign login tokens
`CORS_ORIGINS`	the API	Which website is allowed to call it

Three real problems hit while deploying — and the fixes

PROBLEM 1

The live site was secretly calling the laptop

The website worked on the developer's own machine and was blank for everybody else. Cause: `NEXT_PUBLIC_API_URL` had not been set on the deployed site, so it fell back to `127.0.0.1:8000` — which means "this computer". It found a backend, because the laptop was running one. Fix: set the variable to the real API address and redeploy. **These settings are baked in when the site is built, so changing one always needs a redeploy.**

PROBLEM 2

The API answered every single address with "Not Found"

A configuration file told Vercel to send every request to one entry point. Vercel changed how that works: it now passes the *rewritten* address to the app, so FastAPI received the same internal path for every request and matched nothing. Fix: delete the rewrite, because Vercel detects FastAPI on its own. **The build log said exactly this — reading logs is the skill.**

PROBLEM 3

The API could not reach the database at all

Supabase's direct database address is **IPv6-only** for new projects, and Vercel's functions could not resolve it. Fix: connect through Supabase's **connection pooler**, which offers an IPv4 address. It is also the right choice anyway — serverless functions start and stop constantly, and the pooler shares a small set of database connections between them instead of each one opening its own.

STILL TO DO

Row Level Security

Supabase reports that Row Level Security is switched off on all twelve tables. Our backend connects as the table owner, which bypasses RLS, so switching it on costs nothing and closes a door that is otherwise open. Run `ALTER TABLE public.<name> ENABLE ROW LEVEL SECURITY;` for each table.

11 When something goes wrong

WHAT YOU SEE	WHAT IT MEANS AND HOW TO FIX IT
"The kitchen isn't answering"	The frontend cannot reach the backend. Working locally: the backend terminal is not running. On the live site: check `NEXT_PUBLIC_API_URL` — and remember you must redeploy after changing it.
Products missing but the site loads	The tables exist but are empty. Run `python -m app.seed` again.
`/health` says `database: unavailable`	The API is fine but cannot reach PostgreSQL. Check `DATABASE_URL`. For a hosted database it must go through the pooler address, not the direct one.
`python: command not found`	Python is not on your PATH. Reinstall and tick "Add Python to PATH".
`password authentication failed`	The password in `backend/.env` does not match your PostgreSQL password.
`port 3000 is already in use`	The site is already running in another terminal. Close it, or let Next.js use 3001.
Login stops working after a restart	`SECRET_KEY` changed, which invalidates every issued token. Sign in again.
CORS error in the browser console	The backend is refusing the frontend's address. Add it to `CORS_ORIGINS` and redeploy the API.
Chrome "Local Network Access" warning	A deployed page is calling `localhost`. Same cause as row one.

THE FIRST THING TO CHECK, ALWAYS

Open **kai-ruchi-api.vercel.app/health**. If it says `"database":"connected"`, the backend and database are both fine and the problem is in the frontend. If it does not load at all, the backend is down.

12 Questions your teacher will ask

Answer in your own words — but these are the points that matter.

Why split the frontend and backend instead of one program?

So each can be changed, tested and deployed independently, and so the rules live somewhere the user cannot edit. The same backend could serve a mobile app later.

How are passwords stored?

They are not. Each is hashed with **bcrypt**, which is one-way. On login we hash what was typed and compare hashes. bcrypt also adds a random salt, so two people with the same password get different stored values.

How does the site know I am logged in?

On login the backend issues a **JWT** signed with a secret key. The browser stores it and sends it with every request. If anyone edits the token the signature stops matching and it is rejected.

What is Alembic, and why not create tables by hand?

Alembic records every schema change as a file, so the database can be rebuilt identically anywhere with one command, changes can be undone, and the history sits in version control. Creating tables by hand is not repeatable.

What stops someone editing the price in their browser?

The browser never sends prices — only product IDs and quantities. The backend looks up the real price and recalculates the total. It also re-checks stock at that moment, so two people cannot buy the last jar.

Is the payment real?

No, deliberately — a real gateway needs a registered business. The checkout runs a clearly-labelled simulation that collects no card details at all. The Razorpay integration is written, including HMAC signature verification, and activates only with API keys.

How does order tracking work with no admin panel?

Each status change is a row in `order_events`, and the timeline reads that history. With no staff to click the buttons, a paid order advances on a timer over about 35 minutes so it can be demonstrated live.

How do you know a review is genuine?

On submission the backend checks whether that user has an order containing that product; if so it is flagged **verified purchase**. A database constraint also allows only one review per person per product.

Is it responsive?

Yes, and it was tested by measuring the real page at 375 pixels rather than by eye. Two genuine bugs were found: form fields under 16px made iOS Safari zoom the whole page on focus, and the quantity buttons were below the 44-pixel touch target. Both fixed.

Where is it deployed, and how do the parts find each other?

The website and API are two separate Vercel projects; the database is PostgreSQL on Supabase. They find each other through environment variables — the website is told the API address, the API is told the database address — kept in Vercel's settings so no password ever reaches GitHub.

Did anything go wrong while deploying?

A great question to be ready for. Three things. The live site was falling back to `localhost`, so it only worked on our own machine. The API returned "Not Found" for every address because a rewrite rule was handing FastAPI the same internal path every time. And the database was unreachable because Supabase's direct address is IPv6-only, so we connected through its pooler instead — which is the correct choice for serverless anyway.

What was the hardest part?

Making the cart work for both guests and logged-in users, and merging one into the other at sign-in without losing or duplicating items.

What would you add next?

An admin dashboard so orders and stock can be managed properly, real payments, email notifications, automated tests, and Row Level Security on the database.

13 Where everything lives

If you are asked to open a file and show something, this is the map.

```
kai-ruchi/
├── README.md                  overview of the whole project
├── DEPLOYMENT.md              how to put it on the internet
├── docs/                      this guide
│
├── backend/                   THE PYTHON API
│   ├── alembic/versions/      the migration files (database history)
│   ├── main.py                entry point Vercel looks for
│   ├── app/
│   │   ├── core/
│   │   │   ├── config.py      all settings, read from .env
│   │   │   ├── database.py    the database connection
│   │   │   ├── security.py    password hashing + JWT tokens
│   │   │   └── deps.py        shared checks (who is logged in, whose cart)
│   │   ├── models/            the 12 database tables as Python classes
│   │   ├── schemas/           validation for data in and out
│   │   ├── routers/           the endpoints — auth, products, cart,
│   │   │                      orders, payments, reviews, contact
│   │   ├── services/          the rules — cart totals, order timeline, payments
│   │   ├── seed.py            the starting catalogue
│   │   └── main.py            where the app starts
│   └── requirements.txt       the Python packages needed
│
└── frontend/                  THE WEBSITE
    ├── scripts/               image compression
    ├── public/
    │   ├── images/            all 21 photographs (WebP)
    │   └── videos/            the hero film and four category films
    └── src/
        ├── app/               one folder per page
        │   ├── page.tsx           home
        │   ├── products/          listing + one page per product
        │   ├── categories/        the five shelves
        │   ├── cart/  checkout/   basket and payment
        │   ├── orders/  track/    history and tracking
        │   ├── login/  signup/  account/
        │   └── about/  contact/
        ├── components/
        │   ├── layout/        navbar, footer, loading screen, smooth scroll
        │   ├── ui/            buttons, form fields, animations, kolam
        │   ├── product/       product cards, filters, reviews
        │   ├── cart/          the slide-out basket
        │   └── checkout/      the demo payment sheet
        ├── lib/
        │   ├── api.ts         every call to the backend, in one file
        │   └── types.ts       the shape of the data
        └── store/             cart and login state
```

IF ASKED "SHOW ME THE CODE"

Open `backend/app/routers/cart.py` — short, readable, and it shows validation, a stock check, an error message and a database write all in one place. Then open `frontend/src/lib/api.ts` to show how the frontend calls it.
